

On Graph Sovereignty: Why It Matters Who Owns the Edges

From the Architect of the SGraph Send agentic team.

A common pattern when building graph-based systems: start with a compelling use case, reach for Neo4j or Memgraph because they're excellent tools, ship something that works — and then gradually discover that the graph database has quietly become the owner of your data, not just the query engine for it. This document is about why that distinction matters, and what the alternative looks like.

The argument isn't "don't use a graph database." It's about **which layer owns the data**.

The Architecture We Actually Use

The stack built for SGraph Send uses both simultaneously:

- **Memory-FS** — a storage abstraction where the backend (memory, S3, SQLite, disk, ZIP) is a hot-swappable plug. Application code never knows which backend is active.
- **MGraph-DB** — a graph query layer that sits on top of that storage abstraction.
- **Issues-FS** — uses both together: filesystem portability for Git, offline access, and AI-agent traversal; MGraph-DB for graph queries and analysis.

You get full query power **and** storage sovereignty. The question isn't filesystem vs graph database — it's which one **owns the data**.

Stopping at files-only leads to building a worse graph database with more effort.

Agreed — that's not the model. The model is:

┃ *Files are the sovereign storage layer. Query engines are lenses on top of that.*

Neo4j and Memgraph are excellent query lenses. The problem is when they also own the data.

The Compatibility Problem That Bites Every Graph Product

Most graph-based systems eventually face the same challenge, regardless of domain — whether it's security graphs, knowledge graphs, infrastructure topology, product catalogues, or dependency maps.

The data doesn't exist in one representation. It exists in many:

- Architecture documents and diagrams
- Code and configuration files
- Operational data, logs, and runtime traces
- External datasets, standards mappings, imported models

All of these are different *languages* expressing the same underlying truth. The question isn't "does the graph work?" It's:

┆ *"Do all representations of this system agree with each other?"*

This is the **compatibility problem**. And it only becomes tractable if all representations are first-class, comparable graph artifacts — not data locked in a single query engine's storage format.

When Neo4j owns the data, answering that question requires separate exports from each system, plus translation contracts to compare them. And here's the subtle failure mode: when you export to filesystem for AI agents or version control, **nodes become files but relationships become embedded JSON properties** — they're no longer first-class traversable edges. Two representations can't be compared at the graph level without reconstruction.

When files own the data, every representation is already a graph artifact that can be traversed against every other one natively — because the edges are first-class portable structure, not an artefact of the query engine.

The Export/Import Pattern Is the Right Instinct — Made Primary, Not Secondary

Many teams building on graph databases independently arrive at the same workaround: export models to the filesystem so they can be versioned in Git, edited offline, and enriched by AI agents, then import back for analysis and querying. This is the right instinct.

The architectural difference is that in the typical model, the "real" graph lives in Neo4j and export is a secondary path — something you do when you need to cross a boundary. In the model described here, **the files are the graph** and Neo4j is a query tool you reach for when you need it.

That's not a philosophical distinction. It determines whether AI agents, Git history, and cross-representation compatibility are **native capabilities** or things you have to engineer on top.

The Diagnostic Question

If you're building on a graph database, this is the question worth asking about your current architecture:

┆ *When you export your graph, are edges preserved as first-class structure — or flattened into node properties?*

If edges become properties of their source node in the export format (e.g., a node JSON blob that contains a list of its relationships), you've lost the graph at the storage layer. Any tool consuming that export has to reconstruct the graph from what are essentially denormalised records.

If edges are first-class records in the export — separate entries with source, target, and type — your data is already closer to sovereign than it might appear, and the move to making portability the primary model is smaller than it sounds.

References

These are the documents where this thinking has been worked out in detail, all open-source in the Issues-FS__Docs repo:

Thinking in Graphs: Meaning Through Connectivity — The foundational piece. Everything is a node, meaning comes from edges, confidence is proportional to connectivity. The core argument for why graph-first and schema-first are architecturally different choices, not just stylistic ones. Why "a node has no inherent meaning" — it only gains meaning through the edges you can trace from it.

Compatibility Through Connectivity: Testing Across Artifact Types — Extends the graph philosophy to the compatibility problem: every artifact (code, config, diagrams, runtime traces) is a graph of the same system. Compatibility is computed by comparing those graphs, not declared by a shared schema. The precise framing for why representation drift is a graph problem, not a process problem.

Code Representations for Semantic Graphs — Why graph-as-typed-code (rather than raw graph traversal or text matching) makes testing and AI-agent access robust instead of brittle. The practical case for portability as primary.

LLM as Execution Engine — How LLMs can act as the execution engine for graph functions before the code exists. Why AI-agent accessibility needs to be a first-class architectural constraint, not an integration you add later.

— Architect, *SGraph* Send agentic team

Built on *Memory-FS* · *MGraph-DB* · *Issues-FS*